ROADSAATHI · TOWING & ROADSIDE ASSISTANCE PLATFORM

Product & Developer Handbook

What is built, requirements, architecture, data model, conventions, security, testing and deployment

Audience	For product and engineering
Version	1.0 · 4 November 2026
Source	Generated from the Markdown in `docs/` of the TowingService repository

> Confidential. Third-party figures are cited; RoadSaathi projections are planning assumptions. Legal and tax points must be confirmed by qualified advisers.

Contents

01 Feature inventory — what is built and what is next

02 Product requirements document (PRD) — RoadSaathi

03 Architecture

04 Developer guide

05 Data model

06 Security baseline

07 Testing strategy

08 Contributing

09 Deployment

10 Tracking plan

Feature inventory — what is built and what is next

Status legend: Built **Built** (in `main`, deployed) · Partial **Partial** (works, with a known limitation) · Not built **Not built**. Value legend for future items: **H** high · **M** medium · **L** low.

1. Customer experience (public site, `/[locale]/(site)`)

Feature	Status	Where in code	Notes
Home, services (9), vehicle types (6), pricing, about, FAQ, contact	Built	`app/[locale]/(site)/*`	All copy editable in admin, EN + HI
Coverage page and per-locality SEO pages	Built	`areas/`, `areas/[slug]`	JSON-LD structured data; NCR cities "coming soon" with notify-me
Bilingual UI (English / Hindi)	Built	`src/i18n/en.ts`, `hi.ts`	Typed dictionaries: Hindi must match English keys
Booking wizard (vehicle → service → locations → contact & confirm)	Built	`shared/client/booking-wizard.tsx`, `modules/bookings`	Map search, route distance, scheduling
Instant quote with GST breakdown	Built	`modules/pricing/engine.ts`	Base, per km, minimum, night, surge, rounding, GST
OTP sign-in (email; phone via Firebase with email fallback)	Built	`modules/auth`	Hashed codes, attempt limits, rate limits
Trip PIN (4-digit) to start the job	Built	`Booking.startPin`	Driver must enter it at pickup
Live tracking page and lookup by code	Built	`track/[code]`, `api/track/[code]`	5-second polling
Cancel booking	Partial	`cancelBookingAction`	Cancellation fee is shown but not charged automatically
Rate and review driver	Built	`reviewAction`	Updates driver rating average
Printable GST invoice	Built	`invoice/[code]`, `modules/invoicing`	FY-sequenced numbers, CGST/SGST/IGST
Account page (profile, bookings)	Built	`account/`	
Contact form / call-back leads	Built	`submitLeadAction`	Visible in Admin → Leads
Online payment (Razorpay: UPI, cards, netbanking)	Partial	`modules/payments`, `api/webhooks/razorpay`	Built; shown as "Coming soon" until keys and live mode are enabled

Feature	Status	Where in code	Notes
Refunds	Not built (H)	—	`PaymentStatus.REFUNDED` exists; no refund action or Razorpay refund call
Vehicle photo upload / condition report	Not built (H)	—	Foundation for AI-2 and damage disputes
Consent capture for Terms/Privacy at booking	Not built (H)	—	Legal requirement (chapter 3)
Cookie / analytics consent banner	Not built (M)	—	Google Analytics loads when an ID is set
Account deletion and data export	Not built (H, by May 2027)	—	DPDP data-principal rights
WhatsApp booking and updates	Not built (H)	—	See AI-1
Native Android/iOS app	Not built (M)	—	The site is a PWA (manifest and icons built)
Masked calling between customer and driver	Not built (M)	—	Reduces leakage (risk R3)

2. Partner experience (driver app `/driver`, fleet portal `/partner-portal`)

Feature	Status	Notes
Partner landing page and application (independent driver or company)	Built	`partner/`, `partner/apply`
Admin review, commission setting, approve/reject/suspend	Built	`admin/providers`
Contract templates (versioned), personalised agreement, e-sign with email OTP	Built	Immutable `ProviderContract.renderedBody`; time, IP and method stored
Online/offline toggle, live GPS, capabilities (vehicle types, services)	Built	`api/driver/state`, `api/driver/location`
Job offers with countdown, accept/decline	Built	`modules/dispatch`
Job flow: start driving → arrived → PIN → in progress → complete	Built	State machine in `modules/bookings/state.ts`
Cash collection prompt	Built	
Earnings ledger (gross, commission, net, settled)	Built	`driver/earnings`, `partner-portal/earnings`
Fleet company: add/remove drivers, add trucks, payout details	Built	`partner-portal/*`
Company notified when its driver is assigned	Built	`modules/notifications/events.ts`

Feature	Status	Notes
KYC document upload and verification (DL, RC, insurance, permit, fitness)	Not built (H)	Expiry fields exist on `Truck`; no file storage yet
Push notifications for offers	Not built (H)	Polling today; needs Web Push or a native app
Automated bank/UPI payouts	Not built (M)	Settlement is marked manually with a UTR today
Aadhaar eSign option for agreements	Not built (M)	Recommended in chapter 3
Partner earnings forecast and demand heatmap	Not built (M)	AI-4, AI-15

3. Operations back-office (`/admin`)

Feature	Status	Notes
Dashboard (live counts, recent activity)	Built	
Bookings list and detail with full event timeline	Built	Manual status change, cancel, retry dispatch
Dispatch board (open jobs, online drivers, manual assign)	Built	
Partners management	Built	
Users and staff roles (customer, driver, company owner, dispatcher, admin, super admin)	Built	Invite staff, activate/deactivate
Catalog: services, vehicle types, rate cards	Built	Generic resource editor
Content: areas, FAQs, testimonials, legal pages, contract templates	Built	Markdown, EN + HI, `{{brand}}` placeholders
Settings: brand, contact, business/GST, pricing, dispatch, payments, maps, notifications, SEO	Built	Live, no redeploy
Leads	Built	
Payouts and ledger, settle with reference	Built	
Invoices list	Built	
Analytics funnel (first-party events)	Built	`docs/tracking-plan.md`
Notification log (every email/SMS)	Built	
Audit log (who changed what)	Built	
Grievance register with 48-hour SLA tracking	Not built (H)	Legal requirement; leads table can be a stop-gap
Damage-claim workflow	Not built (H)	Today handled outside the system

Feature	Status	Notes
B2B accounts, API keys, monthly invoicing	Not built (H)	Needed for fulfilment contracts
Reports export (CSV) for CA (GST, TDS/TCS)	Not built (M)	

4. Platform and engineering

Capability	Status	Notes
Modular monolith, Next.js 16 App Router, TypeScript, Prisma 7, Postgres	Built	`docs/ARCHITECTURE.md`
Unit tests: pricing engine, booking state machine, utilities (29 tests)	Partial	No integration or end-to-end tests yet
Deployments: Vercel (primary), Render (Docker), Neon Postgres	Built	Auto-deploy from `main`
Docker Compose local stack (db, mail catcher, migrate, app, cron)	Built	
Cron tick for dispatch timeouts and scheduled releases	Built	Self-advances on tracking polls if cron is late
Security headers, rate limits, Zod validation, signed webhooks	Built	`docs/developer/SECURITY.md`
Error monitoring and alerting (Sentry or similar)	Not built (H)	
CI pipeline (type-check, tests on every PR)	Not built (H)	
180-day log retention (CERT-In)	Not built (H)	Needs a log drain
Realtime (SSE/WebSockets) and Redis for locations	Not built (M)	Scaling path in ARCHITECTURE.md
Self-hosted or paid maps/geocoding	Not built (H before scale)	Provider interface exists (`modules/geo/providers.ts`)
Marketing demo-film pipeline	Built	`marketing/`

Product requirements document (PRD) — RoadSaathi

Owner: product (founder) · **Status:** v1 built; v1.1 launch scope below · **Last updated:** November 2026

1. Vision

Make getting help on the road in India as predictable as booking a cab: **see the price, see the truck, trust the driver, get a proper invoice.**

2. Goals and non-goals

Goals (next 12 months)	Non-goals (for now)
Reliable 24×7 towing and on-spot help across Delhi, then NCR	Owning a truck fleet (we stay asset-light)
Median arrival under 35 minutes in active zones	Statutory/police towing of illegally parked vehicles
Fair, transparent pricing with GST invoicing	Vehicle repairs and spare-parts retail
A partner network operators prefer over listing sites	Insurance products (we partner with insurers, we don't underwrite)
B2B fulfilment for fleets, dealers, insurers and RSA companies	Pan-India launch before NCR unit economics are proven

3. Personas

Persona	Context	Needs	Success looks like
Priya, 32, car owner, Saket	Car won't start at 10:30 p.m. after dinner	Someone trustworthy, fast; knows the cost; safe	Truck arrives in 25 min, price as quoted, invoice on email
Rakesh, 45, owns one flatbed, Mayapuri	Gets 3–4 jobs a day from calls; idle in afternoons	More jobs, fair pay, no chasing payment	2 extra jobs a day, weekly settlement, good ratings
Anil, fleet owner, 8 trucks	Runs a small towing company with employed drivers	Keep trucks busy, see which driver did what, GST paperwork	Every job tracked in the portal; company notified on each assignment
Dispatcher on night shift	Watches open bookings	Know which jobs are stuck, assign manually, call drivers	No job sits unassigned more than 5 minutes
B2B client (insurer / RSA company)	Needs a Delhi network for its policyholders	API, SLA, tracking link, claim evidence	Jobs submitted by API, status webhooks, monthly invoice

4. Core user journeys (all built)

1. **Book:** choose vehicle → service → pickup (and drop for towing) → see quote → sign in with OTP → confirm → get code and PIN.
2. **Dispatch:** nearest eligible online drivers within the radius receive offers (N per round, timeout) → the first to accept is assigned atomically → customer, driver and company are notified.
3. **Fulfil:** driver starts driving → arrives → enters the customer's PIN → job in progress → completes → cash collected or online payment → invoice issued and ledger entry written.
4. **Partner onboarding:** apply → admin approves and sets commission → personalised agreement generated → e-sign with OTP → active.
5. **Settle:** ledger shows what each partner owes or is owed → finance settles with a reference.

5. v1.1 launch scope (must ship before public launch)

ID	Requirement	Acceptance criteria	Priority
L1	**Terms and Privacy acceptance at booking and partner sign-up**	Checkbox (unticked by default) with links; booking or application cannot be submitted without it; store document slug, version, timestamp and IP per user	Must
L2	**Final legal pages**	Lawyer-reviewed Terms, Privacy, Cancellation & Refund, Grievance, Cookie notice; template banner removed	Must
L3	**Grievance officer details**	Name, email, phone and address from Admin → Settings rendered on the grievance page and footer	Must
L4	**Partner KYC documents**	Upload DL, RC, insurance, permit and fitness; admin can view, approve or reject per document; expiry dates stored; partner cannot go online with an expired document	Must
L5	**Analytics consent**	Google Analytics loads only after consent; first-party events contain no PII (already true)	Must
L6	**Production integrations**	DLT SMS live, production SMTP with SPF/DKIM, `EXPOSE_DEV_OTP` off, strong secrets	Must
L7	**Error monitoring and log retention**	Sentry (or similar) alerts on server errors; logs shipped to storage kept 180 days	Must
L8	**Soften e-signature wording** in contract templates (chapter 3) and bump the template version	New version active; old signed contracts unchanged	Must
L9	**Grievance register**	Complaints have status, owner and timestamps; dashboard shows any complaint older than 48 h without acknowledgement	Should
L10	**Refunds for online payments**	Admin issues a full or partial refund through Razorpay; payment status updates; customer emailed	Should (before Razorpay live)
L11	**CI**	Type-check and tests run on every pull request	Should

6. v1.2–v2 scope (post-launch)

Photo condition reports, push notifications, damage-claim workflow, B2B accounts and API, automated payouts, masked calling, WhatsApp booking, account deletion and data export (DPDP, by May 2027), RSA membership, operator SaaS. See `docs/business/05-ai-roadmap.md` for AI items.

7. Non-functional requirements

Area	Requirement
Performance	Quote < 2 s p95 (includes routing); pages LCP < 2.5 s on 4G mid-range Android
Availability	99.5 % monthly for booking and dispatch; cron delay must not stop dispatch (already self-advancing)
Security	OWASP ASVS L1 baseline; role checks on every server action; no PII in analytics
Privacy	Data minimisation; retention per `docs/operations/OPERATIONS-SOP.md` §7; DPDP-ready by May 2027
Accessibility	WCAG 2.1 AA on customer flows (contrast, labels, keyboard)
Localisation	Every customer string in EN and HI; typed dictionaries enforce parity
Money	Integer paise everywhere; totals always equal subtotal + GST (unit-tested)

8. Open product questions

1. Principal vs. marketplace invoicing (decision #2 in the business plan) changes the invoice template and ledger.
2. Should the customer see the driver's phone number, or only masked calling?
3. Cancellation fee: charge online, add it to the partner's next settlement, or waive it during the pilot?
4. Membership design: number of free call-outs, waiting period, vehicles covered.

Architecture

Style: modular monolith — one Next.js 16 deployable (App Router, TypeScript), one Postgres, strict module boundaries inside `src/modules`. Each module can later be extracted into a service because it talks to others only through its exported service functions and owns its tables.

```
src/
  app/[locale]/…        routes only (thin): (site) public, driver, partner-portal, admin
  app/api/…             health, geo, track, driver state/location, event, cron, razorpay webhook
  modules/              business logic (see below)
  shared/lib            env, db, utils, guards (rate-limit/audit), markdown, localized text
  shared/ui|layout|client   design system, shells, client components
  i18n/                 en.ts / hi.ts dictionaries (typed: hi must match en)
prisma/                 schema, migrations, seed
```

Modules and the data they own

Module	Owns	Public surface
`settings`	`SiteSetting` (brand, contact, business/GST, pricing, dispatch, payments, maps, notifications, SEO)	`getSettings/getBrand/updateSetting` (cached by tag)
`auth`	`User`, `Session`, `OtpChallenge`	`requestOtp/verifyOtp/checkOtp`, `getCurrentUser`, role guards
`catalog`	`VehicleType`, `Service`, `RateCard`	cached readers
`pricing`	– (pure engine + quote service)	`computeQuote` (unit-tested), `quoteBooking`
`geo`	`Area` serviceability	provider interface (OSM ⇄ Google), `checkServiceability`
`bookings`	`Booking`, `BookingEvent`, `Review`	`createBooking`, `changeStatus`, `driverAdvance`, state machine
`dispatch`	`DispatchOffer`	`runDispatchRound`, `acceptOffer`, `assignManually`, `dispatchTick`
`providers`	`Company`, `Driver`, `Truck`, `ContractTemplate`, `ProviderContract`	apply, review, e-sign, fleet management
`payments`	`Payment`	cash recording, Razorpay order/verify/webhook
`invoicing`	`Invoice`, `InvoiceCounter`, `LedgerEntry` (written by bookings)	`issueInvoice`
`notifications`	`NotificationLog`	`events.ts` (what to send on each domain event), channels (SMTP, SMS)
`content`	`Page`, `Faq`, `Testimonial`, `Lead`, `Area` (content view)	cached readers, lead action
`analytics`	`AnalyticsEvent`	`trackServer`, event catalogue

Module	Owns	Public surface
`admin`	– (back-office read model + generic resource registry)	allowed to read across modules; writes go through module services

Rules of thumb: pages and actions call module services; modules don't import from `app/`; `shared/` never imports modules (except UI shells that need brand/auth).

Request flow: a booking

```
Customer — /book ─► quoteAction ─► pricing.quoteBooking ─► geo (route + serviceability) + catalog
rate card ─► engine.computeQuote
          ─► createBookingAction ─► bookings.createBooking (re-quotes server-side, PIN, code) ─►
Booking + event
                                                    └─► dispatch.startDispatch ─► offers to N
nearest eligible drivers (+SMS)
Driver  — /driver ─► acceptOfferAction ─► dispatch.assign (atomic updateMany guard) ─► notify
customer, driver, **company**
        ─► driverAdvanceAction ─► EN_ROUTE → ARRIVED → (PIN) IN_PROGRESS → COMPLETED
                                                  └─ completion: payment record (cash), GST invoice,
ledger entry, emails
Cron /api/cron/tick ─► expire offers, next round / NO_DRIVER_FOUND, release scheduled bookings,
cleanups
```

Booking state machine (`modules/bookings/state.ts`)

`PENDING_DISPATCH → OFFERED → ASSIGNED → EN_ROUTE → ARRIVED → IN_PROGRESS → COMPLETED`; `CANCELLED` until work starts; `NO_DRIVER_FOUND` escalates to dispatchers. Every transition goes through `canTransition()` and writes a `BookingEvent` (audit trail for support).

Partner onboarding & contracts

`Apply (driver|company)` → `PENDING` → admin **approve** (sets commission, renders the active `ContractTemplate` into an immutable `ProviderContract.renderedBody`, emails the partner) → partner **e-signs** (typed name + email OTP; time/IP/method stored) → `ACTIVE` (company drivers activate with their company). Independent drivers can be linked to nobody; company drivers carry `companyId` and inherit the company's agreement. Changing a template creates a new version; signed contracts never change.

Money

- All amounts are **integer paise**. `computeQuote` applies base + per-km, minimum fare, night and surge multipliers, rounding, then GST; totals always equal subtotal + GST.
- Commission = basis points on the **taxable value** (driver → company → default). Cash jobs: provider holds the cash and owes platform commission + GST; online jobs: platform owes provider net. Both appear in **Admin → Payouts & ledger**.

- Invoices: FY-sequenced numbers (`RS/2026-27/000001`), SAC code, CGST+SGST (Delhi supply) or IGST, seller/buyer snapshots.

Security

OTP sign-in (hashed codes, attempt limits, per-identifier/IP rate limits), server-side sessions (hashed tokens, revocable), role checks in layouts *and* every server action, input validation with Zod, CSP/HSTS/frame headers, markdown sanitised by construction, signed webhooks, constant-time comparisons, audit log, staff excluded from analytics, redacted OTPs in logs.

Scaling path (India-wide)

1. **Today:** single region, polling (5 s) for tracking/offers, DB-backed rate limits and cache tags — fine for tens of thousands of bookings/day on Neon + Vercel.
2. **Next:** swap polling for SSE/WebSockets (Vercel Functions support them) and move `lastLat/lastLng` writes to Redis; PostGIS for geo-matching; Neon read replica for admin reports.
3. **Multi-city:** `Area.city` already partitions coverage; add `cityId` to rate cards/settings overrides; per-city dispatch tuning.
4. **Extraction candidates:** `dispatch` (stateful, latency-sensitive) and `notifications` (queue) first — both already isolated behind service functions.

Developer guide

Welcome. This guide gets a new engineer from a fresh clone to a merged change. Read it with `../ARCHITECTURE.md` (module boundaries and data flow) and `DATA-MODEL.md`.

1. The product in one paragraph

RoadSaathi is a towing and roadside-assistance marketplace. **Customers** book on a bilingual website, **drivers and fleet companies** (partners) receive and fulfil jobs in a driver app and partner portal, and **staff** run operations from an admin back-office. One Next.js deployable serves all four surfaces, backed by one Postgres database.

2. Stack

Layer	Choice	Notes
Framework	**Next.js 16** (App Router, React 19, Server Components, Server Actions)	⚠ This Next.js has breaking changes vs. older versions. Read `node_modules/next/dist/docs/` before using an unfamiliar API (see `AGENTS.md`)
Language	TypeScript (strict)	`npm run lint` = `tsc --noEmit`
Database	PostgreSQL via **Prisma 7** (`@prisma/adapter-pg`)	Client generated to `src/generated/prisma` (git-ignored)
Styling	Tailwind CSS 4	Safety-yellow and charcoal design tokens; brand colours come from settings
Validation	Zod 4	Every action input and every settings group
Auth	Own OTP + DB sessions (`jose` only for small signed tokens)	Firebase phone auth optional
Maps	Leaflet + OpenStreetMap (Nominatim, OSRM); Google optional	Provider interface in `modules/geo/providers.ts`
Email / SMS	Nodemailer (SMTP), MSG91 or mock	Everything logged in `NotificationLog`
Payments	Razorpay (feature-flagged)	Webhook at `/api/webhooks/razorpay`
Tests	Vitest	`npm test`
Hosting	Vercel + Neon (primary), Render/Docker (secondary)	`docs/DEPLOY.md`

3. Getting started

```
git clone <repo> && cd TowingService
cp .env.example .env
npm install                       # also runs prisma generate
docker compose up -d db mailpit   # Postgres on 5432, Mailpit UI on http://localhost:8025
npx prisma migrate deploy
SEED_DEMO=true npm run db:seed    # catalog, settings, legal pages + demo partners
EXPOSE_DEV_OTP=true npm run dev   # http://localhost:3000 ; OTP codes shown on screen in dev
```

Sign in at `/en/login` with `ADMIN_EMAIL` (default `admin@roadsaathi.example`) and open `/en/admin`. Demo partner logins are in the README. To try the full loop: book as a customer → sign in as a demo driver in another browser → *Go online* → accept → advance → enter PIN → complete.

Command	What it does
`npm run dev`	Dev server with hot reload
`npm run lint`	Type-check the whole project
`npm test`	Unit tests (Vitest)
`npm run build`	`prisma generate` + production build
`npm run db:migrate`	Create a new migration from `schema.prisma` changes (dev only)
`npm run db:deploy`	Apply migrations (CI / production)
`npm run db:seed`	Idempotent seed; never overwrites admin edits

4. Repository map

```
src/app/[locale]/(site)/      public website + booking, tracking, invoice, legal, account
src/app/[locale]/driver/      driver app (PWA)
src/app/[locale]/partner-portal/  fleet-company portal
src/app/[locale]/admin/       back-office
src/app/api/                  route handlers: health, geo, track, driver state/location, event
beacon, cron, Razorpay webhook
src/modules/<name>/           business logic, one folder per bounded context
    service.ts                the module's public functions (other modules call only these)
    actions.ts                "use server" actions called by UI; thin: auth → rate-limit → service →
track
    *.test.ts                 unit tests next to the code
src/shared/lib/               env, db, guard (rate limit, audit), utils, markdown, localized text
src/shared/ui | layout | client   design system, shells, client components
src/i18n/                     en.ts, hi.ts dictionaries (hi is type-checked against en)
src/proxy.ts                  locale routing (Next 16 "proxy", formerly middleware)
prisma/                       schema, migrations, seed + seed-data (catalog, rate cards, FAQs, legal
pages, contracts)
docs/                         this documentation set
marketing/                    demo-film pipeline (separate package)
```

5. Conventions (please follow)

1. **Module boundaries.** Pages and actions call module services. A module never imports from `app/`. `shared/` never imports a module, except UI shells that need brand or auth. The `admin` module may *read* across modules but *writes* through their services.
2. **Server actions re-check auth.** Layout guards are not enough. Every action starts with `getCurrentUser()` or `assertRole([...])` (`modules/auth/guards.ts`) and returns an `ActionResult` (`{ ok: true, data } | { ok: false, error }`).
3. **Validate every input with Zod** at the module boundary, even when the UI already validates.
4. **Rate-limit public and OTP endpoints** with `rateLimit(key, limit, windowSec)` from `shared/lib/guard.ts`.
5. **Money is integer paise.** Never use floats for rupees. Format only at the edge with `formatINR`.
6. **Every booking status change goes through** `canTransition()` and writes a `BookingEvent`. Never update `Booking.status` directly.
7. **Concurrency:** use a conditional `updateMany` (check the affected count) for races such as two drivers accepting the same offer. See `acceptOffer` in `modules/dispatch/service.ts`.
8. **All customer-visible text exists in English and Hindi.** UI strings go in `src/i18n`; DB content uses the `Localized` `{ en, hi }` JSON shape and is read with `L(value, locale)`.
9. **Brand is a variable.** Never hard-code "RoadSaathi". Use `getBrand()` and `{{brand}}` placeholders in content.
10. **Settings over constants.** Business-tunable values belong in a settings group (`modules/settings/schemas.ts`) with a default, so ops can change them without a deploy.
11. **Caching:** readers use `unstable_cache` with tags (`settings`, `catalog`, `content`, `areas`). Writers call `updateTag(tag)` after a change.

12. **Side effects after the response:** notifications use `defer()` (built on `after()`) so a slow SMTP server never slows a booking.
13. **Analytics:** add new event names to `EVENTS` in `modules/analytics/track.ts` *and* `docs/tracking-plan.md`. Never put PII in event props.
14. **Audit:** staff actions that change data call `audit({ actorId, action, entity, entityId, meta })`.
15. **Comments:** explain *why*, briefly. Match the surrounding density.

6. How-to recipes

Add a setting

1. Add the field with a default to the group's Zod schema in `src/modules/settings/schemas.ts` (`schema.parse({})` must still produce a working configuration).
2. Add the field spec to `src/modules/settings/fields.ts` so it appears in Admin → Settings.
3. Read it with `getSetting("group")` or `getSettings()`. No migration is needed (settings are JSON rows).

Add an admin-editable table (catalog or content)

1. Add the Prisma model and run `npm run db:migrate -- --name <change>`.
2. Register it in `src/modules/admin/resources.ts` (fields, columns, cache tags, optional `prepare`). The generic `admin/manage/[resource]` pages give list, create, edit and delete for free.

Add a booking status or transition

1. Update the enum in `schema.prisma` and migrate.
2. Update `NEXT`, `STATUS_LABEL` and, if drivers trigger it, `DRIVER_NEXT` in `modules/bookings/state.ts`.
3. Extend `state.test.ts`. Check notifications (`modules/notifications/events.ts`) and the tracking UI.

Add a notification

Add a function in `modules/notifications/events.ts` that builds the email with `renderEmail` and sends through `sendEmail` / `sendSms`. Respect the toggles in the `notifications` settings group. Call it with `defer()`.

Add a legal page or contract version

Legal pages: Admin → Content → Legal pages, or add to `pages` in `prisma/seed-data.ts` for new installs. The seed **does not overwrite** existing rows, so update production content through the admin. Contract templates: create a new **version**. Signed contracts keep their immutable `renderedBody`.

Swap the map provider

Implement the `GeoProvider` interface in `modules/geo/providers.ts` and select it in Admin → Settings → Maps (or `MAP_PROVIDER`).

Add an AI feature

Follow `docs/business/05-ai-roadmap.md` §5.3: put the provider behind an interface (like geo), call existing module services rather than writing to tables directly, keep humans in the loop for money and suspension decisions, and audit every automated decision.

7. Git workflow

- Branch from `main`: `feat/<short-name>`, `fix/<short-name>`, `docs/<short-name>`.
- Small pull requests with a clear description. Before pushing: `npm run lint && npm test`.
- Conventional commit prefixes, matching history: `feat(scope):`, `fix(scope):`, `chore(scope):`, `docs(scope):`.
- Migrations are committed with the schema change. Never edit an applied migration; add a new one.
- `main` auto-deploys to Vercel and Render, so merge only green, reviewed changes. See `CONTRIBUTING.md`.

8. Where to look when something breaks

Symptom	First place to look
OTP not received	Admin → Notification log; `SMS_PROVIDER` / SMTP env; Firebase config
Booking stuck in "Finding driver"	Admin → Dispatch board; driver `lastSeenAt` freshness (Settings → Dispatch); cron `/api/cron/tick`
Wrong fare	`modules/pricing/engine.ts` (+ tests), rate card row, pricing settings (night hours, surge)
Settings change not visible	Cache tag not updated, or a 120-second `revalidate` window
Webhook not processed	`RAZORPAY_WEBHOOK_SECRET`; signature check in `modules/payments/service.ts`
Health check failing	`GET /api/health` (DB connectivity)

9. Known technical debt (prioritised)

1. No CI, error monitoring or log retention (PRD L7, L11).
2. Tests cover the pricing engine, state machine and utils only. Add integration tests for booking → dispatch → completion, and for auth guards on every action.
3. Polling (5 s) for tracking and offers; move to SSE/WebSockets with Redis-backed locations at scale.
4. CSP allows `'unsafe-inline'` scripts (needed by inline GA bootstrap and Razorpay). Move to nonces.
5. Public OSM endpoints are fair-use only. Self-host or switch providers before marketing spend.
6. No file storage yet (needed for KYC documents and photos). Pick S3-compatible storage with signed URLs.

Data model

Source of truth: `prisma/schema.prisma` (docs/../prisma/schema.prisma) (29 models). Each block is owned by one module, and other modules reach it only through that module's service functions.

Entity overview

```
User ─┬─< Session                          (auth)
      ├─< Booking >── VehicleType, Service  (bookings / catalog)
      ├── Driver ─┬─< Truck                 (providers)
      │           ├─< DispatchOffer >── Booking   (dispatch)
      │           ├─< ProviderContract >── ContractTemplate
      │           └─< LedgerEntry
      └─< Company ─┬─< Driver, Truck, ProviderContract, LedgerEntry
Booking ─┬─< BookingEvent (audit trail of every status change)
         ├─< Payment, ── Invoice (1:1), ── Review (1:1), ── LedgerEntry (1:1)
VehicleType x── Service via RateCard (unique pair)
```

Tables by module

Module	Model	Purpose	Key fields / rules
auth	`User`	Anyone who signs in	`email` and `phone` unique; `role` ∈ CUSTOMER, DRIVER, COMPANY_OWNER, DISPATCHER, ADMIN, SUPER_ADMIN; `isActive`
auth	`Session`	Server-side session	Only `tokenHash` (SHA-256 of token + secret) is stored; 30-day expiry; IP and user agent
auth	`OtpChallenge`	One-time codes	`codeHash`, `attempts`, `expiresAt`, `consumedAt`; per identifier + IP
shared	`RateLimit`	Fixed-window counters	Atomic upsert in `shared/lib/guard.ts`
settings	`SiteSetting`	One JSON row per group	Keys: brand, contact, business, pricing, dispatch, payments, maps, notifications, seo
content	`Page`	Legal/info pages	`title` / `body` are `{en, hi}` markdown with placeholders; `version`; `isPublished`
content	`Faq`, `Testimonial`, `Area`, `Lead`	Site content, service areas, inbound leads	`Area.status` ACTIVE / COMING_SOON with lat/lng/radius
catalog	`VehicleType`, `Service`, `RateCard`	What can be booked and what it costs	Money in **paise**; `RateCard` unique on (vehicleType, service); `Service.requiresDrop`
providers	`Company`	Fleet partner	Status PENDING → APPROVED → ACTIVE (or REJECTED / SUSPENDED); `commissionBp` override; bank/UPI

Module	Model	Purpose	Key fields / rules
providers	`Driver`	Independent or company driver	`companyId` null = independent; `isOnline`, `lastLat/lastLng/lastSeenAt`; rating aggregates
providers	`Truck`	Recovery vehicle	`registrationNo` unique; `kind`; insurance/permit/fitness expiry dates
providers	`ContractTemplate`	Agreement text per party type	Unique (party, version); new terms = new version
providers	`ProviderContract`	A partner's personalised agreement	`renderedBody` is an **immutable snapshot**; signature name, time, IP, method
bookings	`Booking`	A job	`code` unique; `fare` JSON snapshot; `subtotal`, `gstAmount`, `total` in paise; `startPin`; status machine
bookings	`BookingEvent`	Timeline	Every transition, with actor type/id and optional lat/lng
bookings	`Review`	1–5 rating + comment	One per booking
dispatch	`DispatchOffer`	Offer to a driver	PENDING → ACCEPTED / DECLINED / EXPIRED / CANCELLED; round; distance; expiry
payments	`Payment`	Cash or Razorpay record	`providerOrderId`, `providerPaymentId`, raw payload
invoicing	`Invoice`, `InvoiceCounter`	GST tax invoice, FY sequence	Seller and buyer **snapshots**; SAC; place of supply; CGST/SGST/IGST
invoicing	`LedgerEntry`	Money owed per completed job	gross, commission, net (paise); payee driver or company; PENDING → SETTLED with reference
notifications	`NotificationLog`	Every email/SMS attempt	Status, provider, error; OTPs redacted
platform	`AuditLog`	Who changed what	actor, action, entity, meta, IP
analytics	`AnalyticsEvent`	First-party product events	No PII in `props` (see tracking plan)

Invariants worth knowing

- `Booking.total = subtotal + gstAmount`, always (enforced by the pricing engine and its tests).
- Commission is computed on the **taxable value** (`subtotal`), using driver → company → platform-default basis points.
- An invoice and ledger entry are created exactly once, at completion (`bookingId` is unique on both).
- Signed contracts never change. Template edits create a new version.
- Seller details on an invoice are a snapshot, so later changes to business settings never alter issued invoices.

Personal data map (for privacy and DPDP)

Data	Where	Who sees it	Retention (proposed, confirm with counsel)
Name, email, phone	`User`, `Booking.contact*`, `Lead`	Customer, assigned partner (booking contact), staff	Account life + 3 years; bookings per tax record rules
Pickup/drop addresses and coordinates	`Booking`	Assigned partner, staff	With the booking record
Driver live location	`Driver.lastLat/lastLng` (latest only), `BookingEvent` lat/lng	Customer during an active job, staff	Latest point only; event points with the booking
Partner KYC (licence no., PAN, GSTIN, address)	`Driver`, `Company`	Staff	Partnership + 8 years (tax)
Bank/UPI details	`Driver`, `Company`	Staff (finance)	Partnership + 8 years; restrict access
IP addresses, user agents	`Session`, `OtpChallenge`, `AuditLog`, `ProviderContract`	Staff (security)	Sessions until expiry; logs 180 days (CERT-In); contract signature with the contract
Payment identifiers	`Payment`	Staff (finance)	Per tax record rules
Message content	`NotificationLog`	Staff (support)	180 days

Security baseline

This describes the controls in the code today, what must be added before launch, and how to report a vulnerability. It also serves as the "documented information security programme" expected under the IT Act SPDI Rules (chapter 3).

1. Controls already implemented

Area	Control	Where
Authentication	Passwordless OTP; codes stored hashed; attempt limits; expiry; per-identifier and per-IP rate limits	`modules/auth/otp.ts`, `shared/lib/guard.ts`
Sessions	Random 256-bit token in an HTTP-only cookie; only a SHA-256 hash stored; revocable; 30-day expiry; deactivated users lose sessions	`modules/auth/session.ts`, `modules/admin/actions.ts`
Authorisation	Role checks in layouts **and** inside every server action (`assertRole`); resource ownership checks (e.g. a customer can only pay their own booking)	`modules/auth/guards.ts`, module actions
Input validation	Zod schemas on every action and settings group	`modules/*/service.ts`, `modules/settings/schemas.ts`
Output safety	Markdown renderer escapes HTML by construction; React escaping elsewhere	`shared/lib/markdown.ts`
Transport and headers	HSTS (preload), CSP, `X-Frame-Options: DENY`, `frame-ancestors 'none'`, nosniff, strict referrer, restrictive Permissions-Policy	`next.config.ts`
Webhooks	Razorpay signatures verified with constant-time comparison; cron protected by bearer secret	`modules/payments/service.ts`, `api/cron/tick`
Concurrency	Atomic conditional updates for offer acceptance and status changes	`modules/dispatch/service.ts`
Audit	Staff changes logged with actor, entity and IP	`AuditLog`
Privacy	No PII in analytics; staff excluded from analytics; OTPs redacted in notification log	`modules/analytics/track.ts`
Containers	Non-root standalone image with healthcheck	`Dockerfile`

2. Must add before launch

1. **Error monitoring and alerting** (Sentry or similar), with PII scrubbing.
2. **Log retention for 180 days** in storage we control (CERT-In Directions), with access limited to engineering leads.
3. **Secrets:** strong `AUTH_SECRET` (≥ 32 random bytes) and `CRON_SECRET`; `EXPOSE_DEV_OTP` unset; secrets only in the hosting provider's encrypted env vars; a rotation procedure. Rotating `AUTH_SECRET` signs everyone out.
4. **Least privilege:** only the super admin can grant ADMIN; dispatchers cannot view bank details (review admin pages); MFA on Vercel, Render, Neon, GitHub, Razorpay and email-provider accounts.

5. **Backups:** Neon point-in-time restore enabled; quarterly restore drill.
6. **Dependency hygiene:** `npm audit` in CI; Dependabot or Renovate.
7. **CSP hardening:** move from `'unsafe-inline'` scripts to nonces.
8. **File uploads (when KYC/photos ship):** private bucket, signed short-lived URLs, type and size checks, and malware scanning for documents.

3. Security testing

- Run an authenticated DAST scan (e.g. StackHawk/ZAP) against a staging deployment before launch and after major changes.
- Manual checks: IDOR on `/invoice/[code]` and `/track/[code]` (codes are random and unguessable; confirm no PII leaks beyond what the customer needs), privilege escalation via server actions, rate-limit bypass on OTP endpoints.

4. Incident response

Follow `../operations/INCIDENT-RESPONSE.md` (docs/operations/INCIDENT-RESPONSE.md). Specified incidents must be reported to CERT-In **within 6 hours** of noticing them.

5. Reporting a vulnerability

Email the security contact listed on the website (Admin → Settings → Contact → support email, until a dedicated `security@` address exists). Please include steps to reproduce. We aim to acknowledge within 2 business days. Do not access other users' data or degrade the service while testing.

Testing strategy

Today

- **Unit tests (Vitest, 29 tests):** pricing engine (`modules/pricing/engine.test.ts`), booking state machine (`modules/bookings/state.test.ts`), utilities (`shared/lib/utils.test.ts`). Run with `npm test`.
- **Type safety:** `npm run lint` (`tsc --noEmit`), including Hindi/English dictionary parity.

Target test pyramid

Layer	What	Tooling	Priority
Unit	Pure logic: pricing, state machine, commission/ledger maths, invoice numbering, GST split, templating	Vitest	Keep at 100 % of pure modules
Integration	Module services against a real Postgres: `createBooking` → `startDispatch` → `acceptOffer` (incl. race) → `driverAdvance` → invoice + ledger; partner apply → approve → sign	Vitest + disposable DB (Docker or Neon branch)	**High, next**
Authorisation	Every server action rejects the wrong role and someone else's resource	Vitest integration	**High**
End-to-end	Customer booking, driver app loop, admin settings change, in EN and HI, on a mobile viewport	Playwright (Chromium is available in CI images)	Medium
Non-functional	Load (50 concurrent bookings, 200 online drivers polling), accessibility (axe), security (DAST)	k6 / Artillery, axe, ZAP/StackHawk	Before launch

Rules

1. Every bug fix comes with a test that fails before the fix.
2. Money tests assert exact paise, never approximate values.
3. Time-dependent logic (night surcharge, offer expiry) takes `at: Date` as input, as `computeQuote` already does.
4. Never call real SMS, email or payment providers in tests. Use `SMS_PROVIDER=mock` and Mailpit.

Pre-launch manual test script (abridged)

1. Book each service for car and two-wheeler, day and night; compare the quote with the rate card by hand.
2. Cancel at each status; check the fee display and status history.
3. Two drivers accept the same offer simultaneously; only one is assigned.
4. Wrong PIN is rejected; right PIN starts the job.
5. Complete with cash; check the invoice (GSTIN, SAC, CGST/SGST split, FY number) and the ledger entry.

6. Partner applies → approve → receives email → e-signs → becomes active → company driver gets jobs; company is emailed.
7. Change brand name in settings; check site, emails, invoice and manifest update.
8. Switch language to Hindi on every customer page; no English fallbacks in UI chrome.

Contributing

1. **Read first:** `DEVELOPER-GUIDE.md` (conventions §5) and `../ARCHITECTURE.md`.
2. **Branch** from `main` (`feat/…`, `fix/…`, `docs/…`). Keep pull requests small and focused.
3. **Before pushing:** `npm run lint && npm test`. If you changed the schema, commit the migration.
4. **Pull request description:** what changed, why, how you tested it, screenshots for UI changes (EN and HI, mobile width), and any new settings, env vars or migrations.
5. **Review checklist** (author and reviewer):
   - Server actions check auth and ownership; inputs validated with Zod
   - Money in paise; status changes via `canTransition()` + `BookingEvent`
   - New customer text in both `en.ts` and `hi.ts`; brand not hard-coded
   - Cache tags updated after writes; notifications deferred
   - No PII in analytics props or logs; audit log for staff changes
   - Tests added or updated; docs updated (tracking plan, admin guide, this handbook)
6. **Merging** to `main` deploys to production (Vercel and Render). Merge only after review and green checks.
7. **IP:** by contributing you confirm the work is yours to contribute and is covered by your IP assignment agreement with the company.

Deployment

Option 1 — Vercel + Neon (recommended)

1. **Database:** create a Neon project (region *ap-south-1 Mumbai* if available). Copy the **pooled** connection string → `DATABASE_URL`, the **direct** one → `DIRECT_URL` (used by `prisma migrate`).
2. **Project:** import the repo in Vercel (framework: Next.js). Set env vars from `.env.example`: `APP_URL`, `AUTH_SECRET`, `BRAND_NAME`, `ADMIN_EMAIL`, `DATABASE_URL`, `DIRECT_URL`, SMTP_*, `EMAIL_FROM`, `SMS_PROVIDER`, MSG91_*, `CRON_SECRET`, optional `GOOGLE_MAPS_API_KEY`, RAZORPAY_*. **Do not** set `EXPOSE_DEV_OTP`.
3. **Migrate + seed once** from your machine (or CI):

```
DATABASE_URL="<direct url>" npx prisma migrate deploy
DATABASE_URL="<direct url>" ADMIN_EMAIL=you@company.com npm run db:seed   # no SEED_DEMO in prod
```

4. **Cron:** on Vercel Pro copy `vercel.cron.example.json` to `vercel.json` (per-minute). On Hobby, call `GET /api/cron/tick` every minute from an external scheduler with `Authorization: Bearer $CRON_SECRET`. Dispatch also self-advances when customers poll tracking, so a delayed cron degrades gracefully.
5. **Domain & email:** add your domain; configure SPF/DKIM for the SMTP provider (Amazon SES Mumbai, Resend, SendGrid…).

Option 2 — Docker on a VPS / any container host

```
docker compose up -d --build        # db, mailpit, migrate+seed, app, cron
```

For production, edit `docker-compose.yml`: remove `mailpit` and point `SMTP_*` at your provider, set strong `AUTH_SECRET` / `CRON_SECRET`, use a managed Postgres (or keep `db` with a backed-up volume), and put Caddy/Nginx/Traefik in front for TLS. The image is a non-root Next.js standalone server with a `/api/health` healthcheck. Migrations run via the `migrate` one-shot service (`prisma migrate deploy`).

SMS (MSG91)

Register on DLT, get the OTP template approved, then set `SMS_PROVIDER=msg91`, `MSG91_AUTH_KEY`, `MSG91_SENDER_ID`, `MSG91_OTP_TEMPLATE_ID`. Non-OTP SMS (booking updates) use a DLT *flow* template with one variable → `MSG91_FLOW_TEMPLATE_ID` (`VAR1` = message). Until configured, SMS are logged (mock) and visible in **Admin → Notification log**.

Ops checklist

- Health: `GET /api/health` (200 + DB check). Logs: stdout. Add Sentry/Datadog via their Next.js SDKs if desired.
- Backups: Neon PITR or `pg_dump` for self-hosted.

- Secrets rotation: changing `AUTH_SECRET` invalidates sessions and pending OTPs.
- Public OSM endpoints (tiles, Nominatim, OSRM) have fair-use limits — for real traffic host your own, or switch to Google/Mappls.

Firebase Phone Auth (optional)

1. Firebase console → create/choose a project → **Authentication → Sign-in method → Phone → Enable**.
2. **Authentication → Settings → Authorized domains**: add your production domain (e.g. `towingservice.vercel.app`).
3. **Project settings → Your apps → Web app** and copy the config into these environment variables on Vercel/Render, then redeploy: `FIREBASE_PROJECT_ID`, `FIREBASE_API_KEY`, `FIREBASE_AUTH_DOMAIN`, `FIREBASE_APP_ID`.
4. Behaviour: phone numbers are verified with Firebase (invisible reCAPTCHA). If Firebase cannot deliver the SMS (quota, billing, network) and an email address is known, the code is emailed instead. Email OTP is always available. India SMS on Firebase needs the Blaze plan; the free Spark plan only allows a small number of SMS.

Tracking plan

First-party product analytics, stored in Postgres (`AnalyticsEvent`) and read by **Admin → Analytics**. Code: `src/modules/analytics/track.ts` (catalogue + server helper), `src/app/api/event/route.ts` (browser beacon), `src/shared/client/track.tsx` (client helper).

```
meta:
  product: RoadSaathi (name is configurable)
  version: 1
  destinations: [postgres-first-party]      # add a CDP later by forwarding from trackServer()
  naming: snake_case, object_action          # events: booking_submitted, partner_apply_started
  pii_policy: none-in-events                 # no names/phones/emails/addresses in event props.
user_id only.
  internal_user_policy: exclude-staff        # DISPATCHER/ADMIN/SUPER_ADMIN beacons are dropped
server-side
```

Principles

- **Minimal.** Every event answers a funnel, revenue or ops question. No page-view spam (`page_view` is reserved, not auto-fired).
- **Properties over events** for variants (`booking_completed.payment_method`, not `booking_completed_cash`).
- **Business truth is server-side** (booking, payment, review, contract). Browser events only for intent and CTA clicks.
- **Never** put PII or free text in `props`. Money is integer paise, distance is km.

Events

Event	Fired from	When	Properties (all optional unless noted)
`cta_call_click`	browser	Any tap on a call-now link	`where`: topbar, hero, footer, action-bar, contact, mobile-menu, final
`cta_whatsapp_click`	browser	WhatsApp CTA tap	–
`language_changed`	browser	EN/हिं toggle	`to`: en, hi
`booking_started`	browser	Booking wizard opens	`vehicle`, `service` (slugs from deep links)
`booking_step_completed`	browser	Each wizard step passed	`step`: 0–2
`quote_viewed`	server	Fare quote returned	`service`, `total` (paise)
`otp_requested`	server	Any OTP sent	`channel`: EMAIL, SMS
`login_success`	server	OTP verified	`isNew`, `role`
`booking_submitted`	server	Booking created	`code`, `payOnline`

Event	Fired from	When	Properties (all optional unless noted)
`booking_cancelled`	server	Customer cancels	`bookingId`
`booking_completed`	server	Driver completes job	`payment_method`, `total`, `distance_km`, `service_id`, `has_company`
`payment_succeeded`	server	Razorpay capture verified	`method`, `amount`
`review_submitted`	server	Customer rates driver	`rating` 1–5
`job_accepted`	server	Driver accepts offer	`independent`
`partner_apply_started`	browser	Partner form opened	`type`: DRIVER, COMPANY
`partner_apply_submitted`	server	Application saved	`type`
`contract_signed`	server	Agreement e-signed	`contractId`
`coming_soon_lead`	server	“Notify me” for NCR areas	`kind`, `area`
`razorpay_interest_clicked`	browser	Tap on the disabled “Pay online” card	– (demand signal for enabling Razorpay)

Funnels & KPIs (Admin → Analytics)

1. **Booking funnel:** `booking_started → quote_viewed → booking_submitted → booking_completed`.
2. **Supply funnel:** `partner_apply_submitted → contract_signed` (+ approval time from audit log).
3. **Emergency intent:** `cta_call_click` by `where` — tells you which surface drives calls.
4. **Payment demand:** `razorpay_interest_clicked ÷ booking_submitted` decides when to flip Razorpay from *coming soon*.
5. **Geo demand:** `coming_soon_lead` by `area` ranks the next NCR launch.

Operational metrics (time-to-assign, offer acceptance rate, cancellations, ratings) are computed from the transactional tables (`Booking`, `BookingEvent`, `DispatchOffer`, `Review`), not from events.

Entity traits (stored on the transactional tables, not sent as events)

- **User:** `role`, `locale`, `createdAt`. **Driver:** `status`, `companyId` (independent vs fleet), `ratingAvg`, `ratingCount`. **Company:** `status`, `commissionBp`.
- Snapshot counts (online drivers, open bookings) are queried live; no scheduled sync needed at this scale.

Delta / backlog (intentionally *not* instrumented yet)

Candidate	Why deferred
`page_view` auto-tracking	Volume with little signal; add only when running paid acquisition (then forward to GA via `googleAnalyticsId`).

Candidate	Why deferred
`offer_declined`, `offer_expired`	Already in `DispatchOffer` rows.
Forwarding to a CDP / PostHog / Amplitude	Add inside `trackServer()` and `/api/event`; keep event names unchanged.
Naming migration to `object.action`	Pre-launch rename is cheap if you prefer dotted names; do it once, in `EVENTS`.